



## Atividade avaliativa - Programação Orientada a Objetos

Nome do aluno:

---

### Projeto Prático — A Masmorra de Objetos

Mini-RPG em Python. Vale 6,0 pontos. Entrega: arquivos .py

**O cenário.** Você vai construir o núcleo de um pequeno RPG de combate por turnos. No jogo existem personagens de diferentes tipos (Guerreiro, Mago, Inimigo...) que compartilham coisas em comum (nome, vida, atacar) mas lutam de formas diferentes. Esse cenário é perfeito para exercitar todos os conceitos vistos no curso de uma vez só.

Este projeto integra os quatro módulos da disciplina. Cada conceito abaixo será cobrado e pontuado: classe / objeto, construtor, atributos / métodos, encapsulamento, @property, herança, super(), sobrescrita, polimorfismo, organização em arquivos.

### Regras gerais do jogo (o que vamos modelar)

- Todo personagem tem um nome, uma quantidade de vida (HP) e um valor de ataque.
- Um personagem pode atacar outro: o alvo perde vida igual ao ataque do atacante.
- Um personagem está vivo enquanto sua vida for maior que zero.
- Cada tipo de personagem ataca de um jeito diferente (mensagens e/ou regras distintas).
- A vida não pode ser alterada para um valor negativo nem "por fora" sem passar pelas regras do jogo.

**Tempo e foco.** Você tem o tempo restante da prova ( $\approx 1h15$ ) após a Parte Teórica. Priorize fazer o núcleo funcionando (Etapas 1 a 5) antes de tentar os itens bônus. Um projeto menor que roda e demonstra os conceitos vale mais que um projeto grande quebrado.

### O que entregar

- Os arquivos .py do projeto (veja a organização sugerida adiante).
- Um arquivo main.py que, ao ser executado, demonstra o jogo funcionando (uma pequena simulação de batalha — veja a Etapa 5).
- O código deve rodar sem erros com python main.py.
- Comentários curtos sinalizando onde cada conceito aparece (ex.: # herança aqui).

### Etapa 1 — A classe base Personagem (núcleo + encapsulamento)

Crie a classe Personagem, que servirá de mãe para todos os tipos. Ela deve ter:

- Um construtor recebendo nome, vida e ataque.
- A vida deve ser um atributo privado (ex.: self.\_\_vida).
- Uma property vida (getter) que devolve o valor atual.
- Um setter de vida que impede valores negativos: se vier um valor menor que 0, a vida deve ser fixada em 0 (personagem não tem vida negativa).
- Um método esta\_vivo() que retorna True / False conforme a vida.

- Um método `receber_dano(quantidade)` que reduz a vida (usando a property, para respeitar a regra).
- Um método `atacar(alvo)` que aplica o dano do ataque ao alvo e imprime o que aconteceu.

Esqueleto sugerido (você completa as partes com ...):

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.__vida = vida
        self.ataque = ataque

    @property
    def vida(self):
        return self.__vida

    @vida.setter
    def vida(self, novo_valor):
        ... # impede vida negativa

    def esta_vivo(self):
        ...

    def receber_dano(self, quantidade):
        ... # usa self.vida = ... (passa pelo setter)

    def atacar(self, alvo):
        ... # alvo.receber_dano(self.ataque) + mensagem
```

## **Etapa 2 — Subclasses por herança (herança + super)**

Crie pelo menos duas subclasses de `Personagem`. Sugestões: `Guerreiro` e `Mago`. Cada uma deve:

- Herdar de `Personagem` (relação "é um": um `Guerreiro` é um `Personagem`).
- Ter um atributo próprio que a mãe não tem. Ex.: o `Guerreiro` tem `armadura`; o `Mago` tem `mana`.
- Usar `super().__init__(...)` no construtor para reaproveitar `nome`, `vida` e `ataque`, definindo depois apenas o atributo novo.

## **Etapa 3 — Cada um ataca do seu jeito (sobrescrita)**

Sobrescreva o método `atacar(alvo)` em cada subclasse para dar um comportamento próprio:

- `Guerreiro`: ataque corpo-a-corpo. Ex.: imprime "X golpeia Y com a espada!".
- `Mago`: ataque mágico. Ex.: gasta `mana` e imprime "X lança uma bola de fogo em Y!" (se quiser, sem `mana` o mago ataca mais fraco — fica a seu critério).

**Dica de reuso:** dentro do `atacar()` sobrescrito você pode chamar `super().atacar(alvo)` para aproveitar a aplicação de dano da mãe e só acrescentar a mensagem temática — assim você pratica `super()` em um método comum, não só no construtor.

## **Etapa 4 — O Inimigo e o polimorfismo (polimorfismo)**

- Crie mais uma subclasse `Inimigo` (também filha de `Personagem`), com seu próprio `atacar()` sobrescrito (ex.: "O goblin morde X!").
- Agora escreva uma função independente (fora das classes) chamada `rodada_de_ataques(equipe)` que recebe uma lista de personagens de tipos diferentes e faz cada um atacar o próximo da lista (o último ataca o primeiro), percorrendo com um laço `for`.

**Regra de ouro do polimorfismo:** a função rodada\_de\_ataques não pode usar if para checar o tipo de cada personagem (nada de if isinstance(...) ou if tipo == "mago"). Ela apenas chama personagem.atacar(alvo) e confia que cada objeto sabe atacar do seu jeito. É isso que será avaliado aqui.

## Etapa 5 — Simulação no main.py (integração)

No main.py, monte uma pequena demonstração que prove que tudo funciona:

- Crie ao menos um Guerreiro, um Mago e um Inimigo com nomes e atributos à sua escolha.
- Coloque-os numa lista e chame rodada\_de\_ataques(...) pelo menos uma vez.
- Depois dos ataques, imprima o estado de cada personagem (nome, vida atual, vivo ou não).
- Mostre que o encapsulamento funciona: tente reduzir a vida de alguém a um valor que passaria de 0 e confirme (imprimindo) que a vida parou em 0, não foi negativa.

Exemplo de saída possível (a sua pode variar):

```
Thorin (Guerreiro) golpeia Gandalf com a espada! Gandalf perde 12 de vida.
Gandalf (Mago) lança uma bola de fogo em Goblin! Goblin perde 18 de vida.
Goblin morde Thorin! Thorin perde 8 de vida.
--- Estado final ---
Thorin | vida: 22 | vivo: True
Gandalf | vida: 8 | vivo: True
Goblin | vida: 0 | vivo: False
```

## Organização dos arquivos (módulos)

Separe o código em arquivos, como visto na aula de organização. Sugestão:

```
rpg/
  personagem.py  # classe base Personagem
  guerreiro.py   # class Guerreiro(Personagem)
  mago.py        # class Mago(Personagem)
  inimigo.py     # class Inimigo(Personagem)
  combate.py     # função rodada_de_ataques(...)
  main.py        # a simulação (Etapa 5)
```

Use from personagem import Personagem nas subclasses. Se preferir, é aceitável entregar tudo em um único arquivo — mas a separação correta em módulos vale pontos (ver rubrica).

## Rubrica de avaliação (total: 6,0 pontos)

| Critério                                                                     | Conceito avaliado                     | Pts |
|------------------------------------------------------------------------------|---------------------------------------|-----|
| Classe Personagem com construtor e atributos corretos                        | Classe, objeto, construtor, atributos | 1,0 |
| vida privada + property/setter que barra valor negativo                      | Encapsulamento, @property             | 1,0 |
| Subclasses herdam de Personagem e usam super().__init__ com atributo próprio | Herança, super()                      | 1,0 |
| atacar() sobrescrito de forma diferente em cada tipo                         | Sobrescrita                           | 1,0 |
| rodada_de_ataques trata vários tipos sem if de tipo                          | Polimorfismo                          | 1,0 |

|                                                                                     |                       |     |
|-------------------------------------------------------------------------------------|-----------------------|-----|
| main.py roda sem erros e demonstra todas as regras (inclusive a vida travando em 0) | Integração / execução | 0,7 |
| Organização em módulos e comentários sinalizando os conceitos                       | Organização de código | 0,3 |

### **Itens bônus (opcionais — até +0,5, não ultrapassa 10,0 no total)**

- +0,2 — Adicionar um método especial `__str__` em Personagem para que `print(personagem)` mostre nome e vida de forma amigável.
- +0,2 — Curar: método `curar(quantidade)` que aumenta a vida, mas nunca acima de um valor máximo (use a property para validar).
- +0,1 — Loop de batalha que continua atacando em turnos até sobrar apenas um personagem vivo.

### **Sobre cópia e originalidade**

Você pode reaproveitar ideias dos exercícios feitos em aula (a lógica de Conta, de Animal, etc.), mas o domínio aqui é o RPG: o código deve ser seu e refletir o tema. Trabalhos idênticos entre colegas serão avaliados individualmente por arguição.

### **Checklist final (confira antes de entregar)**

- python main.py roda do início ao fim sem erro.
- Existe ao menos uma classe base e três subclasses (Guerreiro, Mago, Inimigo).
- vida é privada e o setter impede valores negativos (testei isso no main).
- As subclasses usam `super().__init__(...)` e têm atributo próprio.
- `atacar()` está sobrescrito e imprime mensagens diferentes por tipo.
- `rodada_de_ataques` percorre a lista com `for` e não checa tipo com `if`.
- Há comentários curtos marcando onde cada conceito aparece.